Spaced-Repetition Review Mode (SM-2)

This is Claude's idea, proposed as the highest-value next feature after a review of the KuantorFlow codebase. It is tracked as GitHub issue #94.

Summary

KuantorFlow stores flashcards and generates quizzes, but it never tracks *recall* — there is no notion of what a learner should review today or what they are about to forget. This document proposes adding a spaced-repetition review mode built on **SM-2**, the classic scheduling algorithm behind SuperMemo and Anki. It is the single mechanic that most directly serves the app's core mission: long-term retention.

Why it matters

The `flashcards` table today holds only content — word, part of speech, explanations, translations, topic, and `created_at`. Quizzes are generated ad hoc over an entire topic, so every card is treated the same regardless of how well the learner knows it. The result is that review effort is spread evenly instead of being concentrated on the material that is slipping.

Spaced repetition fixes exactly this. Well-known cards are pushed out to exponentially longer intervals, while cards the learner fumbles keep returning frequently. Time is spent where forgetting is about to happen. This is not a duplicate of anything currently in the backlog, which is mostly UI, settings, and logging work.

What SM-2 is

SM-2 (SuperMemo 2, published by Piotr Wozniak in 1987) decides *when* to show a card again based on how well it was recalled. It is simple, battle-tested, and the right first implementation.

Per-card state

Each card carries three values:

- `ease` — the easiness factor, starting at `2.5`. Higher means intervals grow faster.
- `interval_days` — how many days until the card is next due.
- `reps` — the count of consecutive successful recalls.

The grading scale

On each review the learner grades their recall with four buttons, mapping to a quality score `q` on the classic 0 to 5 scale:

Button	Meaning	Quality q	Effect
Again	Failed to recall	0 to 2	Reset progress, card returns soon
Hard	Recalled with difficulty	3	Small interval, ease nudged down

Good	Recalled correctly	4	Normal interval growth
Easy	Recalled effortlessly	5	Longer interval, ease nudged up

The update rule

On each review with grade `q`:

- On failure (`q` below 3): reset `reps` to `0` and `interval_days` to `1`, so the card is seen again soon.
- On success: the first success sets the interval to `1` day, the second to `6` days, and every later success sets `interval_days = round(interval_days * ease)`.
- Then adjust ease with `ease = ease + 0.1 - (5 - q) * (0.08 + (5 - q) * 0.02)`, floored at `1.3` so struggling cards never collapse to zero.
- Finally set `due_at = today + interval_days`.

The net effect is that a well-known card follows a widening schedule — roughly 1, 6, 15, 37 days and onward — so almost no time is spent on material the learner already knows.

Proposed scope

Schema

Add review-tracking columns to `flashcards`: `reps INT DEFAULT 0`, `interval_days INT DEFAULT 0`, `ease FLOAT DEFAULT 2.5`, `due_at DATE`, and `last_reviewed_at TIMESTAMP NULL`. Include an idempotent `ALTER TABLE` note in `schema.sql`, matching the existing pattern used for the `pos` column.

Logic

Implement the SM-2 update in `utils.py`: given a card and a grade, recompute `ease`, `interval_days`, `reps`, and `due_at`. Keeping the math in one importable helper makes it easy to unit-test and easy to swap later.

Route

Add `GET /review`, which serves cards where `due_at` is on or before today (or that have never been reviewed) one at a time, and `POST /review/<card_id>`, which records the grade and advances to the next due card.

UI

Add a *Review (N due)* entry point on the index page, reusing the existing flashcard flip styling so the review screen feels native to the app.

Tests

Extend the `kuantorflow_automation` suite with offline tests for the SM-2 math and for the due-selection query, following the existing pattern of importing `app` with the database and network stubbed.

Deployment note

After the change is merged, run the `ALTER TABLE` statement against the PythonAnywhere MySQL database so the new columns exist in production.

A note on newer algorithms

SM-2 is the proven baseline. Anki now defaults to a newer scheduler called FSRS, and there are minor SM-2 variants in circulation. SM-2 remains the right starting point because it is simple and well understood; because the algorithm lives behind a single helper, the formula can be replaced later without touching the schema or the routes.